

COMPILER DESIGN PE - 2

PES2UG23CS311

MAANYA BHARDWAJ

E SECTION

PE2:

Develop a YACC/Bison parser to validate the syntax of a simplified subset of the C language.

The grammar must support:

- Multiple variable declarations with basic types (int, float, char, double) and comma-separated declarators.
- if, if-else, and do-while control statements with support for nested blocks { ... }.
- Arithmetic and relational expressions.

Input: Any C-like source program conforming to the above subset (from stdin or a file).

Output:

- On success: Syntax valid.
- On failure: Syntax error at line <n>, token <t>: <message>

Assumptions:

- Use a LEX/FLEX lexer to provide tokens (ID, NUM, IF, ELSE, DO, WHILE, operators, punctuation).

Ignore comments and whitespace in the lexer.

Lexer.l

```
%{  
#include "parser.tab.h"
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int line = 1;
char current_token[100];
%}

digit [0-9]
letter [_A-Za-z]

%%

"int"    { strcpy(current_token, yytext); return INT; }
"float"  { strcpy(current_token, yytext); return FLOAT; }
"char"   { strcpy(current_token, yytext); return CHAR; }
"double" { strcpy(current_token, yytext); return DOUBLE; }

"if"     { strcpy(current_token, yytext); return IF; }
"else"   { strcpy(current_token, yytext); return ELSE; }
"do"     { strcpy(current_token, yytext); return DO; }
"while"  { strcpy(current_token, yytext); return WHILE; }

{letter}{letter}{digit}* {
    strcpy(current_token, yytext);
    return ID;
}

{digit}+ {
    strcpy(current_token, yytext);
    return NUM;
}

"=="    { strcpy(current_token, yytext); return EQ; }
"!="    { strcpy(current_token, yytext); return NE; }
"<="    { strcpy(current_token, yytext); return LE; }
">="    { strcpy(current_token, yytext); return GE; }
"<"     { strcpy(current_token, yytext); return LT; }
">"     { strcpy(current_token, yytext); return GT; }

```

```

"+"    { strcpy(current_token, yytext); return PLUS; }
"-"    { strcpy(current_token, yytext); return MINUS; }
"*"    { strcpy(current_token, yytext); return MUL; }
"/"    { strcpy(current_token, yytext); return DIV; }
"="    { strcpy(current_token, yytext); return ASSIGN; }

"("    { strcpy(current_token, yytext); return LP; }
")"    { strcpy(current_token, yytext); return RP; }
"{"    { strcpy(current_token, yytext); return LB; }
"}"    { strcpy(current_token, yytext); return RB; }
";"    { strcpy(current_token, yytext); return SEMI; }
","    { strcpy(current_token, yytext); return COMMA; }

```

```

[ \t\r]+ ;
\n      { line++; }

```

```

. {
    strcpy(current_token, yytext);
    return yytext[0];
}

```

```
%%
```

```

int yywrap() {
    return 1;
}

```

Parser.y

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern int line;
extern char current_token[100];

```

```
int yylex();
void yyerror(const char *s);
%}
```

```
%token INT FLOAT CHAR DOUBLE
%token IF ELSE DO WHILE
%token ID NUM
%token PLUS MINUS MUL DIV ASSIGN
%token EQ NE LT GT LE GE
%token LP RP LB RB SEMI COMMA
```

```
%start program
```

```
%%
```

```
program
: declarations statements
;
```

```
declarations
: declarations declaration
|
;
```

```
declaration
: type declarator_list SEMI
;
```

```
type
: INT
| FLOAT
| CHAR
| DOUBLE
;
```

```
declarator_list
: declarator
| declarator_list COMMA declarator
;
```

declarator

- : ID
- | ID ASSIGN expression

;

statements

- : statements statement

|

;

statement

- : expression_stmt
- | selection_stmt
- | iteration_stmt
- | compound_stmt

;

compound_stmt

- : LB statements RB

;

expression_stmt

- : expression SEMI

| SEMI

;

selection_stmt

- : IF LP expression RP statement
- | IF LP expression RP statement ELSE statement

;

iteration_stmt

- : DO statement WHILE LP expression RP SEMI

;

expression

- : assignment

;

```
assignment
: ID ASSIGN assignment
| relational
;
```

```
relational
: relational EQ additive
| relational NE additive
| relational LT additive
| relational GT additive
| relational LE additive
| relational GE additive
| additive
;
```

```
additive
: additive PLUS term
| additive MINUS term
| term
;
```

```
term
: term MUL factor
| term DIV factor
| factor
;
```

```
factor
: LP expression RP
| ID
| NUM
;
```

%%

```
void yyerror(const char *s) {
    printf("Syntax error at line %d, token '%s': %s\n",
        line, current_token, s);
}
```

```
int main() {
    if (yyparse() == 0)
        printf("Syntax valid\n");
    return 0;
}
```

RESULT:

```
cd: no such file or directory: cd_pes2
[maanyabhardwaj@Maanyas-MacBook-Air ~ % cd cd_pe2
[maanyabhardwaj@Maanyas-MacBook-Air cd_pe2 % nano lexer.l
[maanyabhardwaj@Maanyas-MacBook-Air cd_pe2 % nano parser.y
```

```
[maanyabhardwaj@Maanyas-MacBook-Air cd_pe2 % nano parser.y
maanyabhardwaj@Maanyas-MacBook-Air cd_pe2 % bison -d parser.y
flex lexer.l
[gcc lex.yy.c parser.tab.c -o parser
parser.y: conflicts: 1 shift/reduce
[maanyabhardwaj@Maanyas-MacBook-Air cd_pe2 % ./parser < test.c
Syntax valid
[maanyabhardwaj@Maanyas-MacBook-Air cd_pe2 % ./parser < invalid.c
Syntax error at line 2, token 'int': syntax error
maanyabhardwaj@Maanyas-MacBook-Air cd_pe2 %
```

Valid Test File:

```
int a, b;
int x = 10;

do {
    a = a + 1;
} while (a < 10);

if (a == 10) {
    b = 5;
} else {
    b = 0;
```

```
}
```

Invalid Test File

```
int a, b  
int x = 10;
```